

Sistema de Divisão de Despesas

Justificativa do modelo de dados inicial

Análise da adequação das entidades Usuario, Grupo, ParticipanteGrupo, Despesa, Pagamento e ParticipacaoDespesa para a primeira versão do sistema.

Documento técnico - MVP web com Python e Django

Índice

Clique em um tópico para ir diretamente à página em que ele começa.

1. O modelo representa corretamente o problema real
 2. A despesa fica no centro da modelagem
 3. O modelo separa usuários de participantes
 4. O modelo permite participantes sem conta
 5. O modelo suporta várias despesas em cada grupo
 6. O modelo suporta participantes diferentes em cada despesa
 7. Armazenar o valor devido torna o modelo flexível
 8. A tabela de pagamento prepara o sistema para evolução
 9. O cálculo dos saldos fica direto
 10. O modelo favorece a integridade dos dados
 11. O modelo evita duplicação desnecessária
 12. O modelo é compatível com o Django ORM
 13. O modelo equilibra simplicidade e crescimento
- Conclusão

1. O modelo representa corretamente o problema real

O modelo é adequado porque suas entidades correspondem diretamente aos fatos do domínio: quem usa o sistema, em qual grupo a divisão ocorre, qual gasto foi realizado, quem desembolsou o dinheiro e quem deve suportar o custo.

Usuario
Grupo
ParticipanteGrupo
Despesa
Pagamento
ParticipacaoDespesa

Distinção central: o pagamento registra quem colocou o dinheiro; a participação na despesa registra quem deve assumir o custo.

Essa separação permite representar situações reais em que uma pessoa paga tudo, mas várias pessoas dividem o valor.

Despesa: Jantar - R\$ 300,00
Pagamento: Ana pagou R\$ 300,00
Participações:
- Ana deve R\$ 100,00
- Bruno deve R\$ 100,00
- Carlos deve R\$ 100,00

2. A despesa fica no centro da modelagem

A despesa é o fato econômico principal. Ela reúne descrição, valor, data, categoria, grupo e observações, além de se relacionar com pagamentos e participantes.

Despesa

- grupo_id
- descricao
- valor_total
- data_despesa
- categoria
- observacao
- criado_por

Organizar o sistema somente em torno do pagamento seria insuficiente, pois o pagamento registra apenas o desembolso. A despesa representa o gasto completo e funciona como ponto de ligação entre todos os demais elementos.

- Permite mais de um pagador no futuro.
- Permite diferentes formas de divisão.
- Permite participantes com valores distintos.
- Permite anexos, categorias e ajustes sem alterar o núcleo.

3. O modelo separa usuários de participantes

Usuario representa quem possui conta e acesso à aplicação. ParticipanteGrupo representa a participação dessa pessoa em um grupo específico.

Essa diferença é necessária porque um usuário pode participar de vários grupos e exercer papéis diferentes em cada um.

Usuario: Carlos

Grupo: Viagem para Recife

Papel: participante

Grupo: Despesas da casa

Papel: administrador

O papel pertence à relação entre usuário e grupo, e não ao usuário de forma global. A tabela intermediária resolve corretamente essa relação muitos-para-muitos.

ParticipanteGrupo

- grupo_id
- usuario_id
- nome_exibicao
- papel
- ativo

4. O modelo permite participantes sem conta

Em uma viagem, festa ou evento temporário, nem todas as pessoas desejam criar uma conta imediatamente. O modelo pode permitir que `usuario_id` seja opcional e usar `nome_exibicao` para identificar o participante dentro do grupo.

Grupo: Viagem para Recife

Participantes:

- Thiago, com conta
- Ana, com conta
- João, sem conta

Isso reduz o atrito no início do uso. Mais tarde, João pode criar uma conta e associá-la ao registro de participante existente, sem perder o histórico das despesas.

Benefício para o MVP: o criador do grupo pode começar a registrar despesas sem depender do cadastro prévio de todos.

5. O modelo suporta várias despesas em cada grupo

A relação Grupo 1 - N Despesa representa corretamente o uso esperado: um grupo reúne várias despesas, enquanto cada despesa pertence a apenas um grupo.

Grupo: Viagem para Recife

Despesas:

- Restaurante
- Combustível
- Hospedagem
- Passeio
- Supermercado

Essa estrutura facilita a listagem do histórico, o cálculo do total gasto, os filtros e o encerramento ou arquivamento do grupo.

Manter cada despesa vinculada a um único grupo também evita que o mesmo gasto altere indevidamente os saldos de contextos diferentes.

6. O modelo suporta participantes diferentes em cada despesa

Nem todos os membros do grupo precisam participar de todas as despesas. A tabela `ParticipacaoDespesa` registra exatamente quem deve arcar com cada gasto.

`ParticipacaoDespesa`

- `despesa_id`
- `participante_id`
- `valor_devido`

Exemplo: em um grupo com Ana, Bruno, Carlos e Daniela, apenas Ana, Carlos e Daniela participaram de um passeio.

Participante	Valor devido
Ana	R\$ 60,00
Carlos	R\$ 60,00
Daniela	R\$ 60,00

Bruno não aparece nessa despesa e, portanto, seu saldo não é alterado.

7. Armazenar o valor devido torna o modelo flexível

ParticipacaoDespesa não registra apenas a presença da pessoa, mas também o valor final que ela deve.

Mesmo que o MVP comece com divisão igualitária, o mesmo campo permite evoluir para divisão por valor, porcentagem, cotas, proporção ou ajustes de centavos.

Participante	Valor devido
Ana	R\$ 100,00
Bruno	R\$ 60,00
Carlos	R\$ 40,00

O cálculo de saldo não precisa conhecer a regra que originou esses valores. Ele apenas soma quanto cada pessoa pagou e quanto deve.

Resultado: a regra de divisão fica desacoplada do cálculo de saldo.

8. A tabela de pagamento prepara o sistema para evolução

Mesmo que o MVP permita apenas um pagador por despesa, uma tabela própria de Pagamento é mais adequada do que guardar somente pagador_id dentro de Despesa.

Pagamento

- despesa_id
- participante_id
- valor_pago

No futuro, a mesma despesa poderá ter vários pagamentos sem alteração estrutural importante.

Despesa: Hospedagem - R\$ 600,00
Ana pagou R\$ 400,00
Bruno pagou R\$ 200,00

Com dois registros de pagamento, o modelo continua coerente. Se o pagador fosse uma única coluna da despesa, seria necessário remodelar o banco para aceitar essa situação.

9. O cálculo dos saldos fica direto

Com pagamentos e valores devidos separados, o saldo pode ser calculado de forma simples e transparente.

Fórmula: saldo = total pago - total devido

Pessoa	Total pago	Total devido	Saldo
Ana	R\$ 300,00	R\$ 150,00	R\$ 150,00
Bruno	R\$ 50,00	R\$ 150,00	-R\$ 100,00
Carlos	R\$ 100,00	R\$ 150,00	-R\$ 50,00

- Saldo positivo: a pessoa deve receber.
- Saldo negativo: a pessoa deve pagar.
- Saldo zero: a pessoa está quitada.

Esse cálculo pode ser implementado em funções Python independentes e alimentado por consultas ao banco.

10. O modelo favorece a integridade dos dados

As relações podem ser implementadas com chaves estrangeiras no Django ORM, impedindo referências a grupos, despesas ou participantes inexistentes.

- Um participante não deve aparecer duas vezes na mesma despesa.
- O valor devido deve ser maior ou igual a zero.
- O valor pago deve ser maior que zero.
- A soma dos pagamentos deve coincidir com o valor da despesa.
- A soma dos valores devidos deve coincidir com o valor da despesa.
- Pagadores e participantes devem pertencer ao mesmo grupo da despesa.

Parte das regras pode ser garantida pelo banco; regras que envolvem somas e relações de negócio podem ser validadas na camada de serviço.

11. O modelo evita duplicação desnecessária

A estrutura mantém cada informação em seu lugar: dados do usuário ficam em Usuario, informações do grupo ficam em Grupo e dados da despesa ficam em Despesa.

Em vez de repetir nomes, e-mails e descrições em cada lançamento, as tabelas se relacionam por identificadores.

- Reduz inconsistências.
- Facilita atualizações.
- Evita nomes escritos de formas diferentes.
- Simplifica consultas e manutenção.

Se o nome de um grupo mudar, por exemplo, basta atualizar um único registro.

12. O modelo é compatível com o Django ORM

Cada entidade pode ser implementada naturalmente como um model Django. As relações são expressas por ForeignKey e os valores monetários por DecimalField.

```
class Despesa(models.Model):
    grupo = models.ForeignKey(
        Grupo,
        on_delete=models.CASCADE,
        related_name="despesas",
    )
    descricao = models.CharField(max_length=150)
    valor_total = models.DecimalField(
        max_digits=12,
        decimal_places=2,
    )
    data_despesa = models.DateField()
```

Essa correspondência direta facilita migrations, consultas, formulários, uso do Django Admin, testes e evolução gradual do banco.

13. O modelo equilibra simplicidade e crescimento

O modelo não tenta implementar todos os recursos possíveis no início. O MVP pode trabalhar com um pagador, divisão igualitária e poucos papéis.

Ao mesmo tempo, a estrutura comporta evoluções previsíveis:

- múltiplos pagadores;
- divisão por porcentagem, cotas ou valores específicos;
- convites e associação posterior de contas;
- confirmação de acertos e comprovantes;
- categorias personalizadas e despesas recorrentes;
- moedas, auditoria e notificações.

Qualidade principal: é simples o bastante para começar, mas não bloqueia as evoluções mais prováveis.

Conclusão

O modelo inicial é adequado porque separa corretamente os diferentes fatos do domínio.

Grupo: define o contexto da divisão.

ParticipanteGrupo: define quem participa desse contexto.

Despesa: registra o gasto ocorrido.

Pagamento: registra quem desembolsou o dinheiro.

ParticipacaoDespesa: registra quem deve assumir o custo.

Essa estrutura permite calcular saldos de forma simples, representar despesas com participantes diferentes, aceitar múltiplos pagadores futuramente e manter os dados organizados.

Fundamento da modelagem: pagar uma despesa e ser responsável por parte dela são fatos diferentes. Separá-los é essencial para o cálculo correto.